

Project Root, Build and Automation

The root folder, CI, and statically served files

EdgePilot AI - Team 15
Documentation set, 23 August 2026
4 documents in this volume

/ - project root

Project: EdgePilot AI, Team 15. **Work package:** shared; each config file belongs to the work package whose tool it configures.

What this folder is

The root holds three kinds of thing and nothing else: the documents that explain the project to a human, the configuration that tells a tool how to build it, and the two data files the application reads at startup. All source lives under `src/`, all tests under `tests/`, all generated proof under `evidence/`.

What happened here

`00-PROJECT-RECORD.md` was added as the single history of the project. It replaces the five separate audit documents the team's version carried at this level (`AUDIT_REPORT.md`, `FINAL_AUDIT_REPORT.md`, `PRODUCTION_AUDIT_REPORT.md`, `REMAINING_PRODUCTION_GAPS.md`, `TEST_DOCUMENTATION.md`), none of which was marked superseded by the others.

Three Sentry configuration files were removed - they configured a service with no DSN behind it. `.dockerignore` was fixed, which is what made `docker build` succeed; `.gitignore` gained a `*.bak` rule and two documented negation rules; next `.config.mjs` grew from 13 lines to 35 to carry the standalone output setting the Dockerfile depends on.

Files

File	What it is
<code>00-PROJECT-RECORD.md</code>	Every fix, the work package it landed in, the structural comparison, and the known gaps. Read this first
<code>README.md</code>	The public entry point - what EdgePilot is and how a user installs it
<code>CONTRIBUTING.md</code>	External contribution guide
<code>AI_USAGE.md</code>	Required submission item: how AI assistance was used per work package, and how the result was verified. One section per work package
<code>LICENSE</code>	Project licence
<code>package.json</code>	Dependencies and the 29 npm scripts. <code>prebuild</code> and <code>pretest</code> regenerate the vision fixtures, so a build and a test run both start from a known dataset
<code>package-lock.json</code>	Exact dependency tree. Committed so CI and a laptop install the same bytes
<code>tsconfig.json</code>	TypeScript configuration. Note <code>target: es5</code> - it is why spreading a <code>Map</code> , <code>Set</code> or <code>typed array</code> is unsafe in this codebase
<code>next.config.mjs</code>	Next.js configuration, including output: <code>'standalone'</code> for the container image
<code>eslint.config.mjs</code>	Lint rules
<code>jest.config.cjs</code>	Test runner configuration
<code>postcss.config.mjs</code> , <code>tailwind.config.ts</code>	Styling pipeline
<code>Dockerfile</code>	Multi-stage container build. Proven working: 154.8 s, with <code>npm run build</code> succeeding inside the image
<code>docker-compose.yml</code>	Local Postgres plus the app

<code>.dockerignore</code>	What the build context excludes. Was the cause of the Docker build failure and is now correct
<code>.gitignore</code>	Includes <code>*.bak</code> and two documented exceptions - the reference llava measurement and the workloads scaffold
<code>.gitattributes</code>	Line-ending normalisation
<code>.env.example</code>	Every environment variable with a comment. The real <code>.env</code> is never committed
<code>benchmark-tasks.json</code>	The controlled task prompts the dashboard offers
<code>device-profile.json</code>	Left over from the removed device module. Referenced only by <code>.dockerignore</code> and two historical evidence files
<code>seed-local-rows.sql</code> , <code>setup-db.sh</code>	Local database bootstrap

Connected folders

- `src/` - everything next `build` compiles.
- `prisma/` - the schema `package.json`'s `db:*` scripts drive.
- `docs/` - the long-form documentation this file summarises.
- `evidence/` - what the `bench:*`, `vision:*` and `eval:matrix` scripts write.
- `scripts/` - the implementations behind those npm scripts.
- `.github/workflows/` - runs `lint`, `test` and `build` from this `package.json` on every push.

Referenced from

These folders point here. Each link below resolves in both directions.

- `.github/`
- `scripts/db/`

.github/ - repository automation

Work package: integration and API surface, with each check defined by the work package whose script it runs.

What this folder is

GitHub's own configuration directory. It holds one thing: the CI workflow.

What happened here

Unchanged from the team's version. It is listed and documented because the checks it runs are the ones section 5.4 of 00-PROJECT-RECORD.md argues are load-bearing - in particular `npm run build`, which type-checks the whole project where Jest type-checks only what it imports.

Files

Subfolder only. See `workflows/`.

Connected folders

- `workflows/` - the CI definition.
- `/` - the `package.json` whose scripts CI runs.

public/ - statically served files

Work package: product UI.

What this folder is

Files Next.js serves verbatim at the site root. Nothing here is compiled, type-checked or linted.

What happened here

Two files were removed: `index.html` and `favicon.svg`. `index.html` was the more important removal - a static `index.html` in `public/` shadows the App Router route at `/`, so the application's real home page (`src/app/page.tsx`) was being served only when that file happened not to match. Having two definitions of the home page in two different systems is the kind of thing that works until it silently does not.

`edgepilot-live.html` was kept. It is a single 283 KB self-contained page - a standalone demo that runs with no server, no build and no database - which is useful precisely because the rest of the project needs all three.

Files

File	What it is
<code>edgepilot-live.html</code>	Self-contained demo page, 283 KB, all CSS and JS inline. Openable from disk

Connected folders

- `src/app/` - where the real routes live, and why `index.html` had to go.

.github/workflows/ - continuous integration

Work package: integration and API surface.

What this folder is

The GitHub Actions definition that runs on every push and pull request against main and dev.

What happened here

Not modified. Worth knowing what it does and does not cover: it runs lint, tests and a build on a clean Ubuntu runner with Node 24, so it catches the class of defect that a local machine with a warm `.next` cache will not. It does **not** run the evidence scripts (`bench:run`, `vision:run:ollama`, `eval:matrix`) - those need a real provider or a real model, which a CI runner does not have.

Note that `pretest` regenerates the vision fixtures, so the CI test step implicitly proves the fixture generator works on a machine that has never seen the dataset.

Files

File	What it is
<code>ci.yml</code>	34 lines. Checkout, Node 24 with npm cache, <code>npm ci</code> , <code>npm run lint</code> , <code>npm test</code> , <code>npm run build</code>

Connected folders

- `/` - the scripts CI invokes live in `package.json`.
- `tests/` - what `npm test` runs.
- `scripts/vision-benchmark/` - the `pretest` fixture generator CI exercises on every run.

Referenced from

These folders point here. Each link below resolves in both directions.

- `.github/`